

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 -
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szkevenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Algoritmus vezérlése

Az az előírás, amely az algoritmus minden lépésére (részműveletére) kijelöli, hogy a lépés végrehajtása után melyik lépés végrehajtásával folytatódjon (esetleg fejeződjön be) az algoritmus végrehajtása.

- Az algoritmusnak, mint műveletnek a vezérlés a legfontosabb komponense.



Vezérlési módok

- A vezérlési mód azt fejezi ki, hogy egyszerűbb műveletekből hogyan építünk fel összetett műveletet és ennek milyen lesz a vezérlése.
- Négy fő vezérlési módot különböztetünk meg:

Szekvenciális Véges sok adott művelet rögzített sorrendben egymás után történő végrehajtása.

Szelekciós Véges sok rögzített művelet közül adott feltétel alapján valamelyik végrehajtása.

Ismétléses Adott művelet adott feltétel szerinti ismételt végrehajtása.

Eljárás Adott művelet alkalmazása adott argumentumokra, ami az argumentumok értékének pontosan meghatározott változását eredményezi.

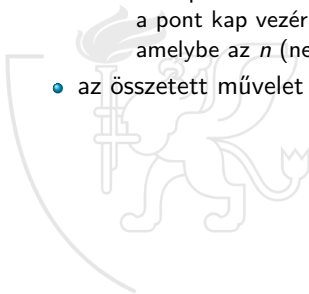
- Az algoritmusok leírására többféle módszer használható
 - Természetes nyelvi leírás
 - Nagyon távol állhat a „géptől”.
 - Pszeudokód
 - „Majdnem” programozási nyelv, struktúrált, de sokkal szabadabb mint egy valódi programozási nyelv.
 - Folyamatábra
 - Grafikus, kevésbé struktúrált (tetszőleges vezérlésátadás), a működési folyamatra koncentrál.
 - Struktogram
 - Grafikus, struktúrált, az algoritmus felépítését is mutatja.

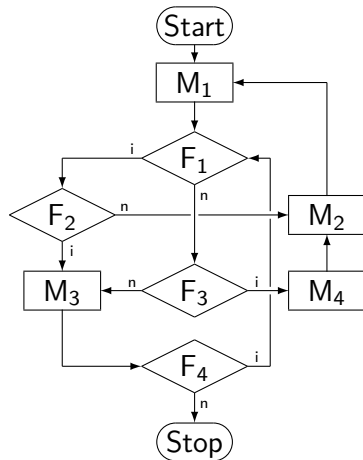
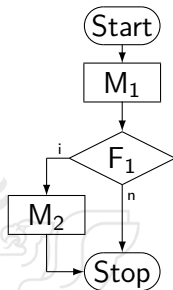
- Ha csak a kész algoritmus működését akarjuk leírni, és a szerkezete kevésbé fontos, akkor használhatjuk a *folyamatábrát*, mint leíró nyelvezetet.
 - Ez olyan ábra, amelyben az algoritmus egyes lépéseit és a lépések közötti vezérlési viszonyt szemléltethetjük (akár különféle alakú) műveletekkel felcímkézett dobozokkal és irányított vonalakkal.
 - Az a mód, ahogyan a folyamatábra leírja az algoritmust nagyon közel áll az alacsony szintű programozáshoz (assembly nyelv).



- A folyamatábra tulajdonképpen egy (vezérlési) gráf grafikus ábrázolása, amelynek
 - van egy (és csak egy) *Start* üres művelettel címkézett pontja, és ebbe a pontba nem vezet él;
 - van egy (és csak egy) *Stop* üres művelettel címkézett pontja, és ebből a pontból nem indul él;
 - minden más pontja egy művelettel vagy feltétellel van címkézve;
 - ha egy pont
 - művelettel van címkézve, akkor belőle egy él indul ki,
 - feltétellel van címkézve, akkor belőle két él indul ki és ezek az *i* (igen) illetve *n* (nem) címkét viselik; és
 - a gráf minden pontja elérhető a *Start* pontból.

- Egy folyamatábra a következő összetett vezérlési előírást jelenti:
 - a végrehajtás a *Start* pontból indul;
 - a gráf egy pontjának végrehajtása azt jelenti, hogy
 - ha a pontban művelet van, akkor a művelet végrehajtódik és a vezérlés a gráf azon pontjára kerül, amelybe a pontból kiinduló él vezet,
 - ha a pont feltétellel van címkézve, akkor kiértékelődik a feltétel, és ha az értéke igaz, akkor az a pont kap vezérlést, amelybe az *i* (igen) címkéjű él vezet, egyébként az a pont kap vezérlést, amelybe az *n* (nem) címkéjű él vezet;
 - az összetett művelet végrehajtása akkor ér véget, ha a *Stop* pont kap vezérlést.

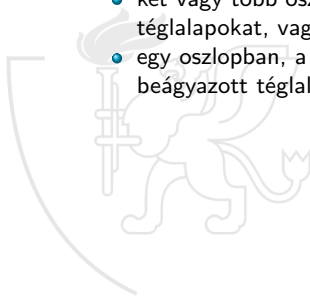




- Ha az algoritmus szerkezete is fontos, akkor használhatjuk a *struktogramot*, mint leíró nyelvezetet.
 - Ez olyan ábra, amelyben az algoritmus egyes lépéseit és a lépések közötti vezérlési viszonyt szemléltethetjük egymásba ágyazott, felcímkézett téglalapok segítségével.
 - A struktogramokkal történő leírás (általában) könnyen átírható imperatív programozási nyelvekre.



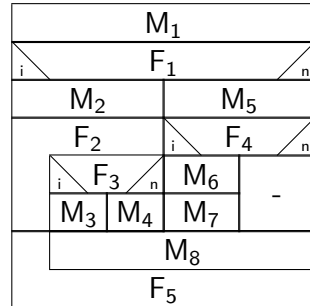
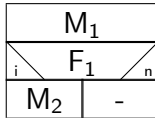
- A teljes algoritmust egy befoglaló téglalap szemlélteti, amelybe egy oszlopban, egymás alá beágyazhatók újabb téglalapok, az alábbiak szerint:
 - ha a téglalap címkéje egy utasítás, akkor a téglalap nem tartalmazhat beágyazott téglalapokat;
 - ha a téglalap címkéje egy feltétel, akkor
 - két vagy több oszlopban, a befoglaló téglalap teljes szélességében tartalmaz beágyazott téglalapokat, vagy
 - egy oszlopban, a befoglaló téglalap jobb oldalán, annak nem teljes szélességében tartalmaz beágyazott téglalapokat.



- Egy struktogram a következő összetett vezérlési előírást jelenti:
 - a végrehajtás a legfelső beágyazott téglalappal indul;
 - ha ez egyetlen utasítással van címkézve, akkor végrehajtjuk az utasítást, és az alatta lévő téglalappal folytatjuk a végrehajtást;
 - ha a legfelső téglalap(ok) címkéje(i) feltétel(ek), és alattuk több oszlop van (a beágyazó téglalap teljes szélességében), akkor a feltétel(eke)t kiértékeljük (balról jobbra, ha több van), és az első olyan oszlop kerül végrehajtásra, amelynek feltétele igaz (vagy az utolsó oszlop, ha egyetlen feltétel sem igaz);
 - ha a beágyazó téglalap egy feltételt és egyetlen beágyazott oszlopot tartalmaz (a jobb oldalon), akkor a feltétel kiértékelése és a beágyazott oszlop végrehajtása a következő szabályok szerint váltakozva ismétlődik:
 - ha a feltétel felül van, akkor az kerül először kiértékelésre, és a beágyazott téglalap csak addig kerül végrehajtásra, amíg a feltétel igaz marad;
 - ha a feltétel alul van, akkor először a beágyazott téglalap kerül végrehajtásra, és utána jön a feltétel kiértékelésre, amíg a feltétel igaz nem lesz;
- a téglalap végrehajtása véget ér, ha elértük az alját és nincs több beágyazott téglalap.

Struktogram

Példa



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Példák (1. verzió)
 - Példák (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Algoritmus vezérlése
 - A Python nyelv utasításai
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés**
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listákhoz tartozó műveletek
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólagos számítás
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Eltelt idő kiszámítása

Problémafelvetés és specifikáció

- Problémafelvetés

- Egy munkahelyen a dolgozóknak minden be- és kilépési időpontot regisztrálni kell. A munkáltató tudni szeretné, ki mennyi időt töltött bent, illetve ebből mennyi volt az ebédszünet. Ehhez kellene neki egy program, amivel a nap két időpontja között eltelt időt tudja kiszámolni.

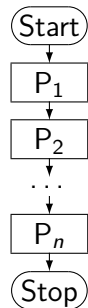
- Specifikáció

- Perc pontossággal dolgozunk.
- A bemenő adat két időpont óra és perc formában, jelöljük ezeket o_1 , p_1 illetve o_2 , p_2 -vel.
- A bemeneti feltétel:
 - $0 \leq o_1 < 24$ és $0 \leq p_1 < 60$
 - $0 \leq o_2 < 24$ és $0 \leq p_2 < 60$
 - $o_1 < o_2$ vagy ($o_1 = o_2$ és $p_1 \leq p_2$)
- A kimenő adat a két bemenő időpont között eltelt idő óra, perc formában, jelöljük ezeket o -val és p -vel.
- A kimeneti feltétel:
 - $0 \leq o < 24$ és $0 \leq p < 60$

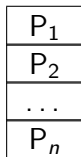
- Számolhatunk kétféleképpen:
 - külön kiszámítjuk az eltelt órákat és perceket, de akkor a végén esetleg korrigálni kell a számítást a negatív percek miatt, vagy
 - eleve percben számolunk, és csak a kiíratáskor alakítjuk át az eredményt óra:perc formátumra.
- Az eltelt idő percben kifejezve $(60 \cdot o_2 + p_2) - (60 \cdot o_1 + p_1)$.
- Tehát o, p akkor és csak akkor megoldás, ha
 - $60 \cdot o + p = (60 \cdot o_2 + p_2) - (60 \cdot o_1 + p_1)$, és
 - $0 \leq p < 60$
- Ez tulajdonképpen a kimeneti feltételek formális megadása, de az első feltétel már mindenképpen az algoritmustervezés fázisához kapcsolódik.

- Szekvenciális vezérlésről akkor beszélünk, amikor a P probléma megoldását úgy kapjuk, hogy a problémát $P_1, \dots, P_n$ részproblémákra bontjuk, majd az ezekre adott megoldásokat (részalgoritmusokat) sorban egymás után hajtjuk végre.
 - $P_1, \dots, P_n$ lehetnek elemi műveletek, vagy nem elemi részproblémák megnevezései. Utóbbi esetben a részproblémát tovább kell bontani.

- Folyamatábrája:



- Struktogramja:



- Az eltelt idő probléma megoldásának struktogramja:

Tájékoztatás
$be(o_1)$
$be(p_1)$
Tájékoztatás
$be(o_2)$
$be(p_2)$
$k = 60(o_2 - o_1) + p_2 - p_1$
$o = k/60$ egészrésze
$p = k/60$ maradéka
$ki(o, p)$



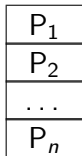
Eltelt idő kiszámítása

Algoritmustervezés – Változók

- Az Eltelt idő algoritmusban használt változók (értékeik egész számok lesznek):
 - $o_1, p_1, o_2, p_2, o, p, k$
- Az egész értékeken a következő műveleteket alkalmazzuk:
 - Összeadás (+)
 - Kivonás (-)
 - Szorzás (*)
 - Osztás egészrésze (//)
 - Osztás maradéka (%)



- Struktogramja:



- Megvalósítása:

A light blue rectangular box with a dark blue border. Inside the box, the processes are listed vertically: P_1 , P_2 , three dots ($\dots$), and P_n .

Eltelt idő

eltelt.py [1–25]

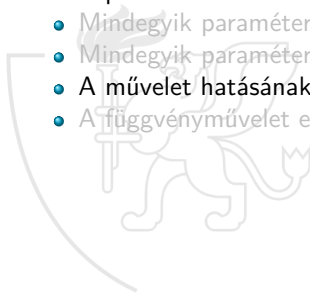
```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Egy nap két időpontja között eltelt idő kiszámítása
9"""
10def main():
11    print("Kérem az első időpontot!")
12    o1 = int(input("óra: "))
13    p1 = int(input("perc: "))
14    print("Kérem a második időpontot!")
15    o2 = int(input("óra: "))
16    p2 = int(input("perc: "))
17
18    e = 60 * o2 + p2 - (60 * o1 + p1)
19    o = e // 60
20    p = e % 60
21
22    print("Az eltelt idő: %d óra %d perc." % (o, p))
23
24if __name__ == "__main__":
25    main()
```

Tartalomjegyzék

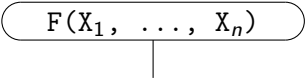
- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Példák (1. verzió)
 - Példák (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szelekciós vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listák adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

- Eljárásvezérlésről akkor beszélünk, amikor egy műveletet adott argumentumokra alkalmazunk, aminek hatására az argumentumok értékei pontosan meghatározott módon változnak meg.
- Az eljárásvezérlés fajtái:
 - Eljárásművelet
 - Függvényművelet
- Vannak olyan programozási nyelvek, ahol a függvény és eljárásműveletek jelentősen meg vannak különböztetve. Mivel azonban e kurzus keretében csak a Python nyelvről lesz szó – ahol a különbség minimális –, egyelőre csak a függvényművelettel foglalkozunk.

- A matematikai függvény fogalmának általánosítása.
- Ha egy részprobléma célja egy érték kiszámítása adott értékek függvényében, akkor a megoldást megfogalmazhatjuk függvényművelettel.
- A függvényművelet specifikációja tartalmazza:
 - A művelet elnevezését
 - A paraméterek felsorolását
 - Mindegyik paraméter adattípusát
 - Mindegyik paraméter kezelési módját
 - A művelet hatásának leírását
 - A függvényművelet eredménytípusát



- A függvényművelet jelölésére a $F(X_1, \dots, X_n)$ formát használjuk, ahol
 - F a függvényművelet neve
 - X_i az i . paraméter azonosítója
- A zárójeleket üres paraméterlista esetén is ki kell tenni.
- A Python jelölésmódhoz igazodva, a függvény algoritmusát egy olyan struktogrammal adható meg, melynek a feje így néz ki:



$F(X_1, \dots, X_n)$

- Továbbá a struktogramon lennie kell (legalább) egy olyan `return` utasításnak, amely visszaadja a függvény által kiszámított értéket.

Függvényművelet

- A fent jelölt függvényműveletnek adott $A_1, \dots, A_n$ aktuális argumentumokra történő végrehajtását függvényhívásnak nevezzük és az $F(A_1, \dots, A_n)$ jelölést használjuk.
- A függvényhívás kifejezés.
- A zárójeleket paraméter nélküli függvény hívása esetén is ki kell tenni.

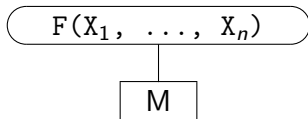


A return utasítás

- Minden függvényben lehet egy vagy több return utasítás.
- Ha a függvényben egy ilyen utasítást hajtunk végre, akkor a függvény értékének kiszámítása befejeződik. A hívás helyén a függvényhívás kifejezés a return által kiszámított értéket veszi fel.
- Ha egy függvényben nincs return utasítás, vagy a függvény nem ennek végrehajtásával ér véget, az olyan, mintha egy return None-t hajtottunk volna végre.



- Struktogramja:



- Megvalósítása:

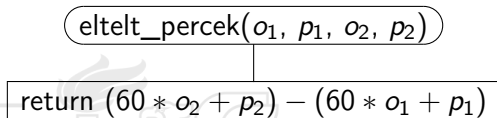
```
def F(X1, ..., Xn):  
    M
```



Eltelt idő kiszámítása

Struktogram

- Az eltelt perceket számító függvény struktogramja:



- Az eltelt idő programjának struktogramja:

Beolvasás
$o = \text{eltelt_percek}(o_1, p_1, o_2, p_2) / 60$ egészrész
$p = \text{eltelt_percek}(o_1, p_1, o_2, p_2) / 60$ maradéka
Kiírás

Eltelt idő

eltelt_fgv.py [1-27]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Egy nap két időpontja között eltelt idő kiszámítása
9"""
10def eltelt_percek(ora1, perc1, ora2, perc2):
11    return 60 * ora2 + perc2 - (60 * ora1 + perc1)
12
13def main():
14    print("Kérem az első időpontot!")
15    o1 = int(input("óra: "))
16    p1 = int(input("perc: "))
17    print("Kérem a második időpontot!")
18    o2 = int(input("óra: "))
19    p2 = int(input("perc: "))
20
21    o = eltelt_percek(o1, p1, o2, p2) // 60
22    p = eltelt_percek(o1, p1, o2, p2) % 60
23
24    print("Az eltelt idő: %d óra %d perc." % (o, p))
25
26if __name__ == "__main__":
27    main()
```